

# Introduction

Vous avez appris à créer des agents capables de suivre des instructions sophistiquées. Ils peuvent répondre de manière intelligente en utilisant l'historique de la conversation : le LLM reçoit automatiquement les messages précédents et comprend le contexte. Cependant, que se passe-t-il si **votre code** doit vérifier des valeurs spécifiques ou prendre des décisions programmatiques basées sur des données de la conversation ?

Exemple :

- Comment vérifiez-vous "L'utilisateur a-t-il fourni son nom ?" dans votre code ?
- Comment stockez-vous une valeur extraite pour pouvoir l'utiliser plus tard dans votre application ?
- Comment prenez-vous des décisions de routage basées sur des données exactes (et non sur une interprétation du LLM) ?

**Limites** : l'historique de la conversation est envoyé au LLM, mais votre code ne peut pas y accéder de manière programmatique. Vous ne pouvez pas écrire `if conversation_history.contains("Alex")` ;, car l'historique de la conversation n'est pas une structure de données que votre code peut interroger. En effet, il s'agit simplement de messages envoyés au modèle.

Ce module présente l'**état de la session**, à savoir un dictionnaire que votre code sait lire et écrire de manière programmatique.

---

## Problème

### L'historique de la conversation ne suffit pas à lui seul pour le contrôle programmatique

Voyons ce qui se passe si vous essayez de créer des agents devant prendre des décisions basées sur des données de la conversation :

```
from google.adk.agents import LlmAgent

# Agent qui doit extraire et mémoriser le nom de l'utilisateur
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extrais le nom de l'utilisateur et mémorise-le pour les questions ultérieures."
)

# Tour 1
# Utilisateur : "Je m'appelle Alex"
# Agent : "Enchanté, Alex !"

# Tour 2
```

# Utilisateur : "Comment est-ce que je m'appelle ?"  
# Agent : "Tu t'appelles Alex"  (le LLM se souvient du nom grâce à l'historique)

# Cependant, dans votre code :  
# Comment vérifier si le nom a été fourni ?   
# Comment accéder au nom exact "Alex" ?   
# Comment l'utiliser dans la logique de l'application ? 

**Problème principal** : Bien que le LLM puisse lire l'historique de la conversation et répondre correctement, votre code ne peut pas vérifier les valeurs de manière programmatique ni prendre de décisions de routage. Vous avez besoin de données structurées, accessibles de manière programmatique.